

Proyecto: red social

Aprendiz: Jhovan D. Cordoba

Programa: ADSO - Análisis y  
Desarrollo de Software

Instructor: JORGE ENRIQUE

PORTELLA CLEVES

Centro De Formación: Sena

Año: 2026

# INTRODUCCIÓN

El control de versiones es una herramienta fundamental dentro del desarrollo de software, ya que permite registrar, organizar y controlar los cambios realizados en los archivos de un proyecto. Su utilización facilita el seguimiento de las modificaciones, la recuperación de versiones anteriores y el trabajo colaborativo entre los integrantes de un equipo de desarrollo.

Para el proyecto de una **red social**, se ha definido previamente una propuesta de interfaz que incluye las pantallas de inicio de sesión, registro, inicio principal, perfil, mensajes y notificaciones. También se estableció una navegación entre estas interfaces mediante un prototipo desarrollado en Figma.

Posteriormente, el diseño front-end contempla el uso de tecnologías como **HTML, CSS y JavaScript**, por lo que resulta necesario contar con una estrategia que permita mantener organizados estos archivos durante el proceso de desarrollo.

En este informe se presentan las herramientas y tecnologías de versionamiento propuestas para el proyecto, así como su función y la manera en que pueden ser utilizadas para controlar la evolución del software.

# OBJETIVO

## Objetivo general

Especificar las herramientas y tecnologías de versionamiento que serán utilizadas para organizar, controlar y conservar las diferentes versiones del proyecto de software **Red Social**, facilitando el seguimiento de cambios durante su desarrollo.

## Objetivos específicos

- Identificar una herramienta adecuada para el control de versiones.
- Seleccionar una plataforma para almacenar el repositorio del proyecto.
- Establecer una estrategia básica para organizar los cambios realizados.
- Facilitar la recuperación de versiones anteriores del software.
- Mantener organizado el código fuente y la documentación del proyecto.
- Aplicar buenas prácticas de versionamiento durante el desarrollo.

# CARACTERÍSTICAS DEL SOFTWARE A DESARROLLAR

El proyecto consiste en el desarrollo de una **red social**, orientada a permitir la interacción entre usuarios.

De acuerdo con las evidencias anteriores, se contemplan las siguientes interfaces:

| Interfaz         | Función                                      |
|------------------|----------------------------------------------|
| Inicio de sesión | Permitir el acceso del usuario               |
| Registro         | Crear una nueva cuenta                       |
| Inicio           | Mostrar publicaciones e interacciones        |
| Perfil           | Consultar información del usuario            |
| Mensajes         | Permitir comunicación entre usuarios         |
| Notificaciones   | Mostrar avisos relacionados con la actividad |

El prototipo anterior establece el flujo:

**Inicio de sesión → Inicio principal**

También permite:

**Inicio de sesión → Registro**

Y desde la pantalla principal:

**Inicio → Perfil / Mensajes / Notificaciones.**

La propuesta visual utiliza una identidad gráfica previamente definida, con colores, tipografía y componentes reutilizables.

## ¿QUÉ ES EL VERSIONAMIENTO?

El versionamiento consiste en llevar un registro organizado de los cambios realizados sobre los archivos de un proyecto.

Por ejemplo, durante el desarrollo de la red social podrían realizarse cambios como:

Versión 1 → Login  
Versión 2 → Registro  
Versión 3 → Pantalla principal  
Versión 4 → Perfil  
Versión 5 → Mensajes  
Versión 6 → Notificaciones

Si posteriormente se presenta un problema en alguna funcionalidad, el historial de versiones permite revisar los cambios realizados y recuperar una versión anterior.

## SELECCIÓN DE HERRAMIENTAS DE VERSIONAMIENTO

Para el proyecto se propone utilizar principalmente:

### **Git**

**Git** será utilizado como sistema de control de versiones.

Permitirá:

- Registrar cambios.
- Crear versiones.
- Crear ramas.
- Consultar el historial.
- Recuperar versiones anteriores.

- Trabajar de forma organizada con el código.

Git será la herramienta principal para controlar los archivos del proyecto.

## **GitHub**

**GitHub** será utilizado como plataforma para alojar el repositorio remoto del proyecto.

Permitirá almacenar:

- Código HTML.
- Archivos CSS.
- Archivos JavaScript.
- Imágenes y recursos.
- Documentación.
- Historial de cambios.

También permitirá disponer de una copia remota del proyecto.

## **Visual Studio Code**

**Visual Studio Code** será utilizado como editor de código para desarrollar el front-end.

En este proyecto permitirá trabajar principalmente con:

HTML  
CSS  
JavaScript

Además, permite integrarse con Git para consultar y administrar los cambios realizados en los archivos.

## **GitHub Desktop**

Como herramienta complementaria se puede utilizar **GitHub Desktop** para administrar Git mediante una interfaz gráfica.

Puede facilitar operaciones como:

- Visualizar cambios.

- Crear commits.
- Crear y cambiar ramas.
- Descargar cambios.
- Subir cambios a GitHub.

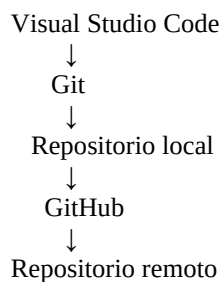
Esto resulta especialmente útil durante el aprendizaje del control de versiones.

## TECNOLOGÍAS DE VERSIONAMIENTO

La tecnología principal seleccionada será:

### **Git + GitHub**

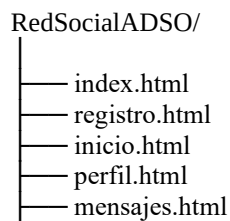
El flujo general será:

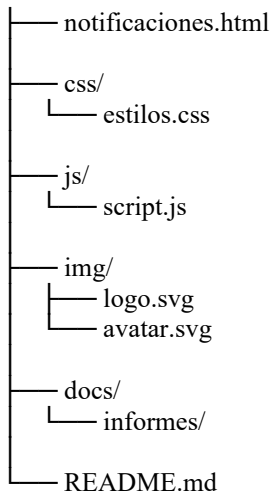


De esta manera, el código desarrollado localmente podrá registrarse mediante Git y posteriormente sincronizarse con GitHub.

## ESTRUCTURA PROPUESTA DEL REPOSITORIO

El repositorio del proyecto podría organizarse de la siguiente manera:





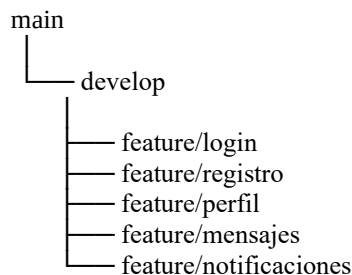
Esta organización permite separar los diferentes componentes del software y facilita identificar los archivos que se modifican durante el desarrollo.

## ESTRATEGIA DE RAMAS

Para organizar el desarrollo se pueden utilizar diferentes ramas.

| Rama      | Función                               |
|-----------|---------------------------------------|
| main      | Contiene la versión estable           |
| develop   | Integra cambios durante el desarrollo |
| feature/* | Desarrollo de nuevas funcionalidades  |
| fix/*     | Corrección de errores                 |

Por ejemplo:



Para una primera etapa académica también se puede trabajar únicamente con main y ramas feature, evitando una estructura innecesariamente compleja.

# COMMITTS

Un **commit** permite registrar un conjunto de cambios realizados en el proyecto.

Por ejemplo:

```
git add .  
git commit -m "Agregar interfaz de inicio de sesión"
```

Otros ejemplos:

feat: agregar pantalla de registro

style: actualizar colores del login

feat: agregar pantalla de perfil

fix: corregir validación del formulario

docs: actualizar documentación del proyecto

Se recomienda que cada commit represente un cambio claro y comprensible.

## FLUJO DE TRABAJO PROPUESTO

El proceso básico sería:

### **Paso 1. Crear el proyecto**

Se crea la carpeta del proyecto y sus archivos iniciales.

### **Paso 2. Crear el repositorio Git**

Se inicializa Git en la carpeta.

### **Paso 3. Realizar cambios**

Se desarrolla una funcionalidad utilizando HTML, CSS o JavaScript.

### **Paso 4. Revisar**

Se verifica que la funcionalidad funcione correctamente.

### **Paso 5. Registrar el cambio**



Se crea un commit.

### **Paso 6. Enviar a GitHub**

Se sincronizan los cambios con el repositorio remoto.

### **Paso 7. Continuar con la siguiente funcionalidad**

Se desarrolla la siguiente parte del proyecto.

## **EJEMPLO DE COMANDOS**

Los principales comandos que podrían utilizarse son:

`git init`

Inicializa un repositorio Git.

`git status`

Permite consultar el estado de los archivos.

`git add .`

Prepara los archivos modificados.

`git commit -m "Agregar interfaz de login"`

Registra los cambios.

`git branch`

Permite consultar las ramas.

`git checkout -b feature/perfil`

Crea una rama para desarrollar una funcionalidad.

`git push`

Envía los cambios al repositorio remoto.

# BENEFICIOS DEL VERSIONAMIENTO PARA EL PROYECTO

La utilización de herramientas de versionamiento permitirá:

- Mantener un historial de modificaciones.
- Evitar la pérdida de avances.
- Recuperar versiones anteriores.
- Organizar el desarrollo de las funcionalidades.
- Facilitar la colaboración.
- Identificar cambios realizados.
- Mantener un respaldo remoto.
- Separar nuevas funcionalidades de la versión estable.

## BUENAS PRÁCTICAS

Para el proyecto se recomienda:

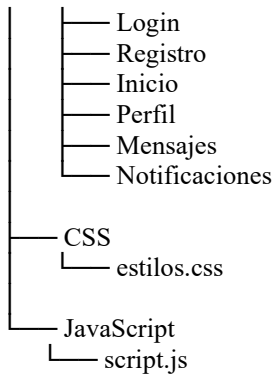
1. Realizar commits pequeños y relacionados con una funcionalidad.
2. Utilizar mensajes descriptivos.
3. Mantener actualizado el repositorio.
4. No almacenar contraseñas ni información sensible.
5. Crear ramas para nuevas funcionalidades.
6. Revisar el código antes de integrarlo a la rama principal.
7. Mantener organizado el proyecto.
8. Documentar los cambios importantes.

## RELACIÓN CON EL PROYECTO FRONT-END

El versionamiento estará directamente relacionado con el desarrollo realizado anteriormente.

Por ejemplo:

Git  
|  
└── HTML



De esta manera, cada modificación realizada sobre las interfaces, estilos o funcionalidades podrá quedar registrada en el historial del proyecto.

# CONCLUSIONES

El control de versiones es un componente importante dentro del desarrollo del proyecto **Red Social tipo Facebook**, ya que permite organizar y conservar los cambios realizados durante la construcción del software.

La utilización de **Git** permitirá administrar las diferentes versiones del código, mientras que **GitHub** servirá como repositorio remoto para almacenar y respaldar el proyecto.

Por otra parte, **Visual Studio Code** permitirá desarrollar los archivos HTML, CSS y JavaScript, mientras que **GitHub Desktop** podrá utilizarse como herramienta gráfica complementaria para facilitar la administración del repositorio.

La implementación de una estrategia de ramas, commits descriptivos y una estructura organizada permitirá mantener un proceso de desarrollo más controlado y facilitará la evolución del software.

# REFERENCIAS

- Git. (s. f.). *Git Documentation*. <https://git-scm.com/doc>
- GitHub. (s. f.). *GitHub Documentation*. <https://docs.github.com/>
- Microsoft. (s. f.). *Visual Studio Code Documentation*.  
<https://code.visualstudio.com/docs>